

武汉理工大学毕业设计（论文）

基于图像处理的 Word 文档格式检查系统的设计 与实现

学院（系）： 计算机与人工智能学院

专业班级： 计算机科学与技术 2204 班

学生姓名： 李冯哲

指导教师： 孙玉芬

学位论文原创性声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包括任何其他个人或集体已经发表或撰写的成果作品。本人完全意识到本声明的法律后果由本人承担。

作者签名：

年 月 日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保障、使用学位论文的规定，同意学校保留并向有关学位论文管理部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权省级优秀学士论文评选机构将本学位论文的全部或部分内容编入有关数据进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

本学位论文属于 1、保密口，在 年解密后适用本授权书

2、不保密口。

（请在以上相应方框内打“√”）

作者签名： 年 月 日

导师签名： 年 月 日

摘 要

论文格式检查是学术写作中的重要环节。针对人工校对效率低、易遗漏等问题，本文设计并实现了一个基于 Python 的 Word 文档格式检查系统。系统以页面版面识别为主线，在 Word 转 PDF 后提取页面文本、坐标及局部空间关系，对页码、页眉、章节分页、图题表题位置和公式编号位置等版面问题进行自动判断。

在此基础上，系统结合 Word 文档对象解析，对摘要、关键词、目录、标题、引文、参考文献以及字体、字号、对齐方式和行距等内容进行辅助检查，并实现了规则调度、结果合并、问题定位和图形界面展示功能。

测试结果表明，系统能够较好地完成多数常见论文格式问题的自动检查，在文档结构检查、样式属性检查、页码页眉检查和结果可视化展示方面具有较好的实用效果。本文完成了一个以页面识别为核心、结合文档对象分析的论文格式检查原型系统。

关键词：格式检查；Word 文档；页面分析；可视化定位

Abstract

Format checking is an important part of academic writing. To improve the low efficiency and omission risk of manual proofreading, this paper designs and implements a Python-based Word document format checking system.

The system takes page-layout recognition as the main line. After converting a Word document into PDF, it extracts page text, coordinates, and local spatial relations to detect page numbers, headers, chapter pagination, caption positions, and formula numbering positions. It also combines Word document object parsing to check abstracts, keywords, tables of contents, headings, citations, references, and style attributes such as fonts, font sizes, alignment, and line spacing. In addition, unified rule scheduling, result merging, issue localization, and graphical presentation are implemented.

The test results show that the system can effectively detect most common paper-format problems and performs well in structure checking, style checking, page-number and header checking, and visual result presentation. This paper completes a prototype paper-format checking system centered on page recognition and supported by document object analysis.

Key Words: Format checking; Word Document; Page Layout Analysis; Visual Localization

目 录

第 1 章 绪论	1
1.1 课题研究背景	1
1.2 课题研究目的与意义	1
1.3 国内外研究现状	2
1.4 本文研究内容	3
1.5 论文结构安排	4
第 2 章 相关技术	5
2.1 Word 文档结构与格式特征	5
2.2 python-docx 文档对象解析技术	5
2.3 PDF 页面文本提取与坐标分析方法	6
2.4 文档页面版面分析相关方法	6
2.5 PyQt 图形界面开发技术	7
2.6 本章小结	7
第 3 章 系统需求分析与总体设计	7
3.1 系统建设目标	7
3.2 系统功能需求分析	8
3.2.1 文档输入与预处理需求	8
3.2.2 文档对象检查需求	8
3.2.3 页面分析检查需求	8
3.2.4 检查结果展示需求	9
3.2.5 用户交互需求	9
3.3 系统规则需求分析	9
3.3.1 基于文档对象的规则需求	10
3.3.2 基于页面版面的规则需求	10
3.3.3 混合检查需求	10
3.4 系统总体处理流程	11
3.5 系统总体架构设计	12
3.5.1 文档对象分析层	12
3.5.2 页面分析层	12
3.5.3 结果处理与展示层	13
3.6 本章小结	14
第 4 章 系统实现	15
4.1 文档对象分析实现	15
4.1.1 文档上下文的构建	15
4.1.2 文档结构与格式信息提取	15
4.1.3 样式继承与特殊对象处理	16
4.2 页面分析与角色识别实现	16
4.2.1 页面对象的构建	17
4.2.2 页面角色识别	17
4.2.3 页面格式规则的实现	17

4.3 规则调度与典型规则实现.....	18
4.3.1 混合检查流程的组织.....	19
4.3.2 文档对象规则与页面规则的分工.....	19
4.3.3 统一问题对象的构建与输出.....	20
4.4 问题定位与可视化实现.....	20
4.4.1 文件导入与检查触发.....	21
4.4.2 检查结果列表与页面联动.....	21
4.4.3 文档对象问题的近似定位.....	21
4.4.4 错误标注框的绘制与缩放.....	22
4.5 本章小结.....	23
第 5 章 系统测试与结果分析.....	25
5.1 测试环境与测试方法.....	25
5.2 测试样本文档设计.....	25
5.3 文档对象侧规则测试结果.....	26
5.4 页面分析侧规则测试结果.....	28
5.5 综合检查结果分析.....	30
5.6 系统不足与误差分析.....	31
5.7 本章小结.....	32
第 6 章 结论与展望.....	33
6.1 论文工作总结.....	33
6.2 主要成果总结.....	33
6.3 不足与后续展望.....	33
参考文献.....	35
致 谢.....	36

第 1 章 绪论

1.1 课题研究背景

随着高校论文写作逐渐全面电子化，论文排版与格式规范问题也变得越来越突出。对于学位论文和课程论文等常见学术文档而言，除研究内容本身外，论文格式是否符合学校统一要求，同样是质量评价中的重要组成部分。这类论文通常包含封面、摘要、目录、正文、参考文献、致谢等多个部分，不同部分在字体、字号、对齐方式、行距、页眉、页码以及图表公式编号等方面均有较为明确的规范要求。如果这些内容依靠人工逐项核对，不仅工作量较大，而且容易因为疏忽而遗漏问题。

从实际写作过程来看，学生在论文撰写中经常会遇到格式设置不统一、目录与正文不一致、页码位置不正确、标题层级错误、图题表题位置不规范、参考文献编号混乱等问题。尤其需要注意的是，在论文经过多次修改以及反复排版处理之后，原本已经调整得比较完善的格式可能会再一次发生变动，这样一来，后期人工校对工作的负担便得到了进一步的加重。所以，去研究一种可以自动发现论文格式问题的辅助检查系统，有着较强的现实意义。

另一方面，近年来文档处理、智能排版分析以及文本信息提取等技术持续发展，这就给论文格式的自动检查工作打下了技术基础。借助 Word 文档对象解析技术，能够较为便利地去获取段落、标题、字体以及样式这些结构化信息；借助 PDF 页面分析技术，则能够进一步获取页面里面的文本位置和版面布局方面的特征信息。把这两类信息进行结合，就有希望实现对论文的结构类规则以及版面类规则的联合检查工作，这样一来，就能提升格式检查的覆盖范围和准确性。

基于上述背景，本文选择论文格式自动检查作为研究对象，设计并实现一个基于 Python 的 Word 文档格式检查系统，并以学校论文规范作为主要应用场景展开验证，以期对论文排版规范化提供一种可行的辅助工具。

1.2 课题研究目的与意义

本课题的研究目的，是围绕论文格式检查这一实际需求，设计并实现一个能够对 Word 文档进行自动分析、自动检查和结果可视化展示的软件系统。系统不仅需要对摘要、关键词、目录、标题、参考文献等内容结构和样式属性进行检查，还需要对页码、页眉、章节分页、图题表题位置和公式编号位置等版面规则进行分析，并通过可配置规则适配不同论文规范，从而尽可能覆盖常见论文中的格式问题。

从研究意义来看，本文首先具有较强的实际应用价值。论文写作几乎是每位高校学生都会接触到的任务，而论文格式检查是一项高频且耗时的工作。若能够通过系统自动发现部分明显格式问题，就可以在一定程度上减轻人工校对压力，提高论文修改效率。与此同

时，本文也具有一定的技术研究意义。论文格式检查并不是单纯的文本比对问题，而是同时涉及页面版面分析、文本位置理解、文档结构解析和结果定位展示等多个环节。本文以页面表示分析为主要切入点，并辅以 Word 对象解析完成规则补充，这对于相关文档智能处理任务也具有一定参考意义。

除此之外，本文还具有一定的工程实现意义。本文不仅关注规则本身的判断，还重视系统整体流程的搭建，包括文档输入、规则调度、结果合并、页面定位和图形化展示等内容。通过完成一个可运行的原型系统，可以更直观地验证相关技术方案的可行性。因此，本文的研究既来源于真实需求，也服务于真实需求，同时兼顾了一定的技术探索与工程实现价值。

1.3 国内外研究现状

近年来，随着文档数字化程度不断提高，文档格式自动检查逐渐成为文档处理与智能审校领域的重要研究方向。现有相关研究主要集中在文档版面分析、文档结构理解、文档要素检测与自动审校等方面。

在面向文档版面分析与文档理解的研究中，国内学者已开展了较为系统的探索。叶栩见^[1]针对文档版面分析中的边缘信息利用不足和多模态协同问题，提出了基于边缘信息引导与可变形注意力的物理布局分析方法，并设计了基于多模态协同的逻辑结构提取方法，为文档结构化理解和版面要素识别提供了新的思路。赵钰豪等^[2]系统回顾了文档解析技术从传统规则方法到深度学习方法的发展过程，重点分析了场景文本检测、表格理解和 LayoutLM 系列多模态预训练模型，指出多模态预训练模型已成为文档解析的重要发展方向。王家宁等^[3]从版面智能布局定义出发，整理归纳了当前基于深度学习的版面智能布局算法，探讨了生成式设计在版面布局中的应用趋势，为版面自动分析与生成提供了方法论支持。刘成林等^[4]对文档智能分析与识别技术进行了较全面的综述，从图像预处理、版面分析到文本识别等多个环节总结了深度学习方法的研究进展，并指出了识别精度、自适应性和可解释性等方面仍存在的问题。

在具体版面分析算法方面，相关研究主要围绕检测精度、模型轻量化和复杂场景适应性展开。蔡云冰等^[5]提出了面向移动端部署需求的轻量化卷积神经网络文档版面分析算法，在保证较少参数量的同时取得了较好的检测效果。张河锁^[6]针对复杂多场景文档版面分析问题，构建并公开了自然拍照手写文档数据集，提出了能够处理多层级分割任务的方法，提升了复杂文档场景下的版面分析能力。蔡云冰等^[7]进一步提出基于参数重分配策略的通用文档版面分析方法，通过多尺度特征提取和轻量化结构设计提高了模型的通用性与准确率。Shehzadi 等^[8]提出了一种半监督文档版面分析框架 DocSemi，在仅使用少量标注数据的情况下仍能保持较高检测性能，为低资源场景下的文档分析提供了有效方案。

尹玲等^[9]针对文档版面分析效率低和训练成本较高的问题，提出了基于改进 YOLOv5s 的文档图像版面分析算法，通过改进特征提取与损失函数设计提升检测效果。袁沛愉等^[10]围绕多模态元素解析难题，提出了融合多尺度特征的文档版面分析方法，以增强复杂版面场景下的识别能力。Minouei 等^[11]研究了将自然场景目标检测器迁移到文档版面分析任务中的可行性，验证了基于目标检测思路处理文档布局问题的潜力。马晓松^[12]提出了基于多模态学习的文档版面分析框架，通过融合图像特征、布局特征和关键字特征提高复杂布局下的分析准确率。谢海龙等^[13]提出了基于复合注意力机制的文档版面分析算法，通过空间注意力、可变形卷积和通道注意力的结合增强了复杂区域检测能力。Zhou 等^[14]则针对像素分类方法难以保持区域连续性的问题，提出了基于位置编码和边界框规范的文档版面分析方法。

国外研究同样在文档版面分析和文档理解方面发展较快。Shehzadi 等^[8]提出的半监督框架为低标注成本场景下的版面分析提供了有效思路。Minouei 等^[11]验证了目标检测方法在文档版面分析任务中的适用性。Zhou 等^[14]则从版面区域连续性保持的角度，对传统像素分类思路进行了改进。

在面向实际应用的文档审校与格式检测研究中，也已有部分成果。谭伯乐^[15]面向公文格式审核需求，提出了基于大语言模型的文档要素检测和语法纠错方法，并结合专门数据集和训练框架实现了较完整的智能审校流程。这类研究说明，文档格式检查不仅可以停留在传统规则匹配层面，也可以与文档检测、文本理解和智能纠错等技术结合起来，形成更完整的文档智能处理系统。

从整体上来看，国内外相关研究已经为文档格式自动检查打下了比较好的理论基础，并且给予了相应的技术支持，特别是在版面分析、文档理解以及文档要素检测这些方面，取得了比较明显的进展。但是，针对于高校毕业论文这一特定的场景，自动化的格式检查系统仍然算是比较少的。现有的方法，或者是偏重于底层的版面分析工作，或者是偏重于通用的文档理解工作，对于毕业论文里面同时包含的结构类规则、样式类规则和页面类规则的综合检查，其支持还是不够充分的。鉴于此，本文尝试把 Word 文档对象分析跟页面分析这两种方法结合起来，去构建一个面向毕业论文格式检查场景的辅助系统，借此来提高对多类型格式问题的覆盖能力，同时让结果展示的直观性得到进一步的提升。

1.4 本文研究内容

围绕论文格式自动检查这一目标，本文的研究内容主要集中在几个相互衔接的方面。首先，本文分析了论文撰写规范中常见的格式要求，并对适合自动检查的规则进行了整理和分类，将其划分为更适合通过 Word 文档对象分析实现的规则，以及更适合通过页面版面分析实现的规则。其次，本文设计了系统总体架构和处理流程，构建了由页面分析层、

文档对象分析层和结果处理展示层组成的系统结构，使页面版面分析与文档结构解析能够在同一系统中协同工作。

在功能实现方面，本文分别完成了文档对象侧规则检查和页面版面侧规则检查。前者主要针对摘要、关键词、目录、标题、引文、参考文献以及字体字号、对齐方式、行距等规则展开，后者则利用 Word 转 PDF 后的页面文本与坐标提取结果，对页码、页眉、章节另起页、图题表题位置和公式编号位置等问题进行分析。在此基础上，系统进一步实现了结果合并、问题定位和图形化展示功能，使不同来源的问题结果能够被统一组织，并在界面中以问题列表和页面标注的形式呈现。最后，本文通过样本文档和异常样例对系统进行了测试与分析，总结了当前系统的有效性、局限性和后续可改进方向。通过上述研究内容，本文完成了一个以论文格式检查为目标、并以学校论文规范作为重点场景的原型系统实现。

1.5 论文结构安排

本文共分为六章，各章内容安排如下。

第 1 章为绪论，主要介绍课题研究背景、研究目的与意义、国内外研究现状、本文研究内容以及论文整体结构安排。

第 2 章为相关技术，主要介绍 Word 文档结构与格式特征、python-docx 文档对象解析技术、PDF 页面文本提取与坐标分析方法、文档页面版面分析相关方法以及 PyQt 图形界面开发技术。

第 3 章为系统需求分析与总体设计，主要从系统建设目标、功能需求、规则需求、总体处理流程和总体架构设计等方面，对系统进行整体分析。

第 4 章为系统实现，重点介绍文档对象分析、基于页面版面识别的格式检查、混合规则调度以及问题定位与图形界面的实现思路。

第 5 章为系统测试与结果分析，主要介绍测试环境、测试方法、测试样本文档设计，并结合系统运行截图与测试结果分析当前方案的优点、不足和误差来源。

第 6 章为结论与展望，总结全文的主要工作和研究成果，并对系统后续可优化的方向进行展望。

第 2 章 相关技术

2.1 Word 文档结构与格式特征

Word 文档是当前论文写作中最常见的电子文档形式之一，其内容不仅包含普通文本，还包含段落、标题、页眉、页脚、表格、图片、公式、脚注、参考文献等多种结构元素。与纯文本文件相比，Word 文档的一个重要特点在于，它同时保存了内容信息和格式信息。例如，同一段文字不仅有文本本身，还可能关联字体、字号、加粗方式、对齐方式、行距、缩进以及所在样式等属性。

对于毕业论文而言，Word 文档中的格式特征具有较强的层次性。论文一般是由封面、摘要、目录、正文、参考文献、致谢等多个部分共同组成的，不同部分在标题的形式、字体的字号和排版的方式等方面往往会存在明显区别。比如，摘要标题、正文的一级标题、参考文献的内容还有页码，在字体字号的要求上一般都并不相同。所以，要是想实现自动化的格式检查，那就需要先去理解 Word 文档内部的结构组织方式以及它的格式特征。

从实现的角度来看，Word 文档中的论文格式问题可大致划分为两类。第一类涉及结构与样式属性方面的问题，例如标题层级是否正确、字体字号是否符合要求、参考文献编号顺序是否规范；第二类涉及页面版面布局方面的问题，例如页码是否居中、页眉是否从指定位置起始、图题表题摆放位置是否正确等。前者更适合从 Word 文档对象本身提取，后者则往往需要结合页面最终呈现结果进行分析。正因如此，本文在系统设计中既使用了 Word 对象分析，也引入了页面分析链路。

2.2 python-docx 文档对象解析技术

python-docx 是 Python 环境里经常选用的一种 Word 文档处理库，可以对.docx 文件当中的段落、样式、表格还有节等对象进行读取以及部分的编辑工作。有关于本文所开展研究的毕业论文格式检查任务，python-docx 借助它较为便捷的结构化访问特性，成为了文档对象分析模块的一个关键基础。

在运用 python-docx 时，开发者可以把 Word 文档看作是由多个对象层次构成的一种结构。比如，文档对象包括了多个段落以及节对象，而每个段落里面又包含了若干个 run 对象，并且每一个 run 对象都可以对应到各不相同的字体和字号属性。借助这些对象，系统就能够获取到段落文本、标题样式、对齐方式、行距还有部分的页眉页脚信息，进而可以对摘要、关键词、目录、标题、参考文献等内容开展格式检查工作。

不过，python-docx 也存在着一定的局限性。它更为适宜去读取文档的逻辑结构以及样式属性，而对于一些底层的比较复杂的内容，比如页码域代码、文本框当中的内容还有最终页面布局信息，仅仅依靠常规对象接口往往是不能完整地获取到的。因此，在本文的实现当中，除了直接运用 python-docx 所提供出来的对象属性之外，还结合底层的 XML 结构，

对部分特殊内容开展了补充分析工作，以此来提高页码、页脚和页眉等区域信息提取的准确性。

总的来看，python-docx 为论文格式检查提供了一个较为良好的文档对象分析基础，尤其适宜于处理标题、段落、字体字号、对齐方式、引文和参考文献等结构与样式属性类规则。

2.3 PDF 页面文本提取与坐标分析方法

对于页码、页眉、图题表题位置和公式编号位置等版面问题，仅通过 Word 文档对象难以准确判断，因此需要引入页面层面的分析方法。本文选择将 Word 文档转换为 PDF，再从 PDF 中提取页面文本及其坐标信息，以此作为页面分析的基础。

PDF 与 Word 文档相比，更接近文档最终输出时的页面呈现形式。通过 PDF 页面文本提取，可以获得每一页的整体文本内容，以及文本块、文本行或文本 span 在页面中的边界框坐标。这些坐标信息能够反映元素在页面中的实际位置，是判断“是否位于顶部”“是否位于底端居中”“是否位于图下表中”等空间关系的重要依据。

在本文系统中，页面分析阶段主要使用两类信息。第一类是整页文本信息，主要用于页面角色识别，例如判断某一页是否为目录页、摘要页或正文页；第二类是局部文本与坐标信息，主要用于页码候选提取、页眉检测、题注定位以及问题可视化标注。通过同时使用这两类信息，系统能够兼顾整体页面识别与局部版面分析。

因此，PDF 页面文本提取与坐标分析方法是本文实现页面规则检查的重要技术基础。

2.4 文档页面版面分析相关方法

文档页面版面分析是指从页面角度研究文档中各类内容的分布方式、空间关系和结构区域。相较于只关注文本内容本身，版面分析更重视文本块、标题、页码、图片、表格和公式等元素在页面中的相对位置。

在论文格式检查任务中，版面分析的意义主要体现在两个方面。其一，某些规则本质上依赖空间关系，例如页码是否位于页面底端居中、图题是否位于图下方、表题是否位于表上方、公式编号是否位于公式右侧行末等。其二，页面整体结构本身也包含较强语义信息，例如目录页通常包含点线连接的目录项，正文首页通常具有“第 1 章”或“绪论”等起始标记，参考文献页和致谢页往往在页面顶部出现明显标题。

本文系统中的页面版面分析方法主要采用基于规则和特征匹配的思路，而不是复杂的深度学习模型。一方面，这是因为当前课题更关注具体规范场景下的可实现性；另一方面，毕业论文的页面结构通常具有较强规律性，利用文本模式、坐标区域和相对位置关系，已经能够支持多数常见版面规则检查。虽然这种方法在复杂文档场景下仍有局限，但对于当前论文格式检查任务具有较好的适用性。

2.5 PyQt 图形界面开发技术

为了使检查结果更直观地呈现给用户，本文采用 PyQt 构建图形用户界面。PyQt 是 Python 环境下常用的桌面界面开发工具，提供了丰富的窗口控件、布局管理机制和事件响应能力，适合实现具有一定交互复杂度的桌面应用程序。

在本系统中，PyQt 主要用于完成以下几类功能。第一，构建文档选择、规则执行和结果展示等基础交互界面；第二，显示 PDF 页面图像及问题标注结果；第三，实现问题列表与页面跳转之间的联动；第四，支持页面缩放、浏览和结果概览展示。

引入图形界面的意义在于，论文格式检查不仅是“有没有发现问题”，还涉及“问题在哪里、是否便于确认”。如果系统只输出文字结果，用户仍需要手动翻找文档对应位置，使用体验会受到较大影响。通过 PyQt 提供的界面展示能力，本文将检查结果、问题列表和页面可视化标注整合到同一界面中，提高了系统的可用性。

2.6 本章小结

本章对本文实现过程中所依赖的主要相关技术与理论基础进行了介绍，其中包括了 Word 文档结构与格式特征、python-docx 文档对象解析技术、PDF 页面文本提取与坐标分析方法、文档页面版面分析相关方法以及 PyQt 图形界面开发技术。这些内容为论文格式检查系统提供了实现的基础，同时也为后续章节展开系统分析与功能实现提供了方法层面的支撑。

第 3 章 系统需求分析与总体设计

3.1 系统建设目标

本课题面向论文格式检查场景，目标是设计并实现一个能够对 Word 文档进行自动检查的软件系统。系统应当可以依据毕业论文、课程论文或其他学术论文的撰写规范，对论文里面的封面、摘要、目录、正文、图表、公式、参考文献等部分开展格式方面的检查工作，并输出相应的错误提示或警告信息，帮助用户找出排版中存在的问题。

传统的 Word 文档格式检查工作大多依靠文档对象本身的结构信息，比如段落、样式、字体、字号、对齐方式等。这类方法对于文本内容以及样式属性类问题，有较好的检查效果，但是在涉及页面布局以及视觉呈现方面的规则时，往往就会存在一定的局限。比如，页码是否处在页面底端居中、页眉是否从第 1 章开始启用了、章节标题是否另起一页、图题和表题的位置是否正确等等问题，仅仅依靠文档对象的属性是很难准确判断的。因此，本系统将页面分析作为主要检查路径，并使用文档对象分析对结构和样式规则进行补充。

结合课题要求与实际实现情况，系统建设目标可以概括为几个彼此关联的方面。首先，系统需要完成对 Word 文档主要结构内容的自动识别与解析，并在此基础上实现文档对象

属性类规则的自动检查。其次，系统还应具备对页面布局类规则的检测能力，以弥补仅依赖文档对象分析时在版面判断上的不足。在结果层面，系统需要将.docx 检查结果与页面分析结果统一整合，并进一步提供问题定位与可视化展示能力。通过这些目标的实现，构建一个兼顾结构信息与页面信息的毕业论文格式检查系统，提高论文格式检查的自动化程度和结果的直观性。

3.2 系统功能需求分析

根据毕业论文格式检查任务的特点，系统需要具备文档输入、格式检查、结果汇总和可视化展示等基本功能。结合当前系统实现情况，可以将功能需求划分为以下几个方面。

3.2.1 文档输入与预处理需求

系统首先应支持用户选择待检查的 Word 文档作为输入对象。由于部分页面规则无法直接依赖 Word 文档对象完成判断，因此系统还需要具备将 Word 文档转换为 PDF 页面表示的能力，或者支持用户直接选择已有的同名 PDF 文件参与页面分析。

在文档输入完成后，系统应分别完成两类预处理：一类是对.docx 文档进行结构解析，提取段落、标题、样式等对象信息；另一类是对 PDF 页面进行文本与坐标提取，为后续页面规则检查和问题标注提供基础数据。

3.2.2 文档对象检查需求

文档对象侧主要面向内容结构与格式属性规则。系统应能够对论文中的主要板块进行检查，包括封面、中文摘要、英文摘要、目录、正文、参考文献和致谢等部分。同时，还应能够进一步检查字体、字号、行距、对齐方式、字符间距、标题编号、关键词数量、摘要长度、参考文献编号顺序、引文上标等具体规则。

这部分功能的重点在于利用 python-docx 获取到的结构化信息，对论文内容本身的规范性进行判断。

3.2.3 页面分析检查需求

页面分析侧主要面向页面布局与视觉呈现规则。系统应能够识别 PDF 页面中的文本块及其坐标位置，并在此基础上完成以下类型的检查：页码位置与连续性检查、页眉启用边界检查、目录页识别、章节是否另起一页、图题表题相对位置检查、公式编号位置检查以及注释位置辅助检查等。

这部分功能的重点在于结合页面中的文本分布、坐标信息和局部区域特征，对论文的版面格式进行分析。

3.2.4 检查结果展示需求

系统不仅要输出检查结果，还要尽量让结果便于理解。为此，系统需要提供统一的问题列表展示能力，能够对不同来源的检查结果进行汇总，并按照一定顺序编号显示。同时，系统还应支持错误位置的页面可视化标注，使用户能够在页面图像中直接看到问题所在区域，提高结果的可读性和实用性。

3.2.5 用户交互需求

在用户交互方面，系统应支持文档选择、执行检查、问题浏览、页面跳转和页面缩放等基础操作。用户点击问题列表中的某一项后，系统应能够切换到对应页面并在左侧显示相关标注，便于快速定位问题。

图 3-1 展示了系统对应的用户用例图。表 3-1 对系统功能需求进行了归纳，用于说明各功能模块的边界与作用。

3.3 系统规则需求分析

论文格式检查规则种类较多，不同规则适合采用的实现方式也不同。为了提高系统的可实现性和检查效果，有必要对规则进行分类分析。

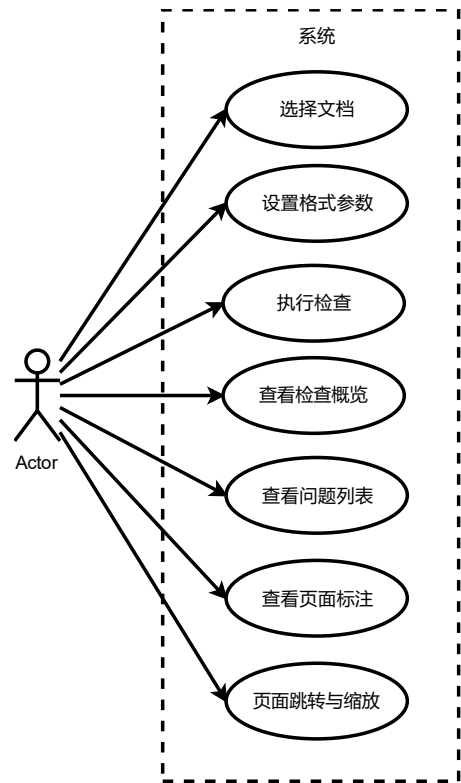


图 3-1 用户用例图

表 3-1 系统功能需求汇总表

功能模块	输入	主要处理	输出	用户可见结果
文档输入模块	.docx 文件	文件选择、路径校验、同名 PDF 查找	文档路径、PDF 路径	已加载待检查文档
文档对象分析模块	.docx 文档对象	板块识别、标题提取、样式提取	结构化上下文数据	无直接界面输出
页面分析模块	PDF 页面数据	文本提取、坐标提取、页面角色识别	页面文本与坐标集合	无直接界面输出
规则执行模块	文档上下文、页面上下文	执行文档对象规则与页面规则	问题对象集合	检查概览与问题列表
结果展示模块	问题对象集合	排序、编号、聚合、格式化显示	结果文本、标注信息	问题列表与统计结果
可视化定位模块	问题对象、PDF 页面	页面跳转、边框标注、联动显示	标注页图像	问题定位与页面高亮

3.3.1 基于文档对象的规则需求

部分规则与文档对象自身的结构和样式属性关系密切，适合通过 Word 文档对象分析实现。例如，摘要字数、关键词数量、标题编号层级、标题标点、字体字号、对齐方式、行间距、参考文献编号顺序、引文上标等，都可以通过提取段落文本和样式属性完成检查。

这类规则的特点是判定依据较明确，对页面视觉布局依赖较小，因此适合作为文档对象侧规则实现。

3.3.2 基于页面版面的规则需求

还有一部分规则，从本质上来说，需要依靠页面最终呈现的效果才能做出准确的判断，仅凭借文档对象很难做到。例如，页码是不是处在页面底端正中间、页眉是不是从第 1 章开始启用、目录页的页面特征、章节标题是不是位于新页的页首、图题和表题的上下位置关系、公式编号是不是处在公式右侧的行末，等等。这些规则更适合借助页面文本块、行位置、坐标范围和局部版面关系来开展判断工作。这类规则是本系统页面分析链路中的重点内容，同时，也是展现出题目里“基于图像处理”这一特点的关键部分。

3.3.3 混合检查需求

在实际论文格式检查过程中，仅依赖单一方式难以覆盖所有规则。因此系统需要以页

面版面分析为主，同时结合文档对象分析，并对两类规则结果进行统一组织。对于页面规则，可直接利用页面号和坐标框进行展示；对于文档对象侧规则，则需要通过文本匹配与页面角色限制等方式进行近似定位，以便在图形界面中实现统一标注。因此，系统还需要具备混合规则调度、结果合并和统一输出的能力。

表 3-2 列出了系统中各类规则与实现方式之间的对应关系。

3.4 系统总体处理流程

根据系统建设目标和功能需求，系统总体处理流程可以概括为一个连续的处理链路，具体如图 3-2 所示。用户首先选择待检查的 Word 文档，系统随后对文档进行结构解析，构建文档对象上下文；与此同时，系统会自动寻找同名 PDF，若不存在则执行 Word 转 PDF，以获得用于页面分析的页面表示。在得到 PDF 后，系统进一步提取页面文本与坐标信息，并分别执行文档对象侧规则和页面分析侧规则。两类规则产生的问题结果最终会被统一汇总、排序，并在图形界面中以检查概览、问题列表和页面标注等形式展示出来。这一流程保证了文档内容检查与页面版面检查能够在同一框架下协同工作，既保留了文档对象分析对样式信息的利用优势，又利用页面分析补足了版面类规则的检查能力。

表 3-2 规则分类与实现方式对应表

规则类别	典型规则	主要数据来源	判断依据	结果展示方式
文档对象 结构规则	摘要长度、关键词数量、目录一致性	.docx 段落与板块 结构	文本内容、板块边界、条目数量	问题列表文本 展示
文档对象 样式规则	字体字号、行距、对齐方式、字符间距	.docx 段落、run、 样式信息	样式属性与规范 值比对	问题列表文本 展示
页面版面 规则	页码位置、页眉内容、章节另起页	PDF 页面文本与 坐标	空间位置、候选区域、页面角色	问题列表与页 面标注
图表公式 规则	图题表题位置、公式编号位置	PDF 局部文本块 与坐标	相对位置关系、局部窗口特征	问题列表与页 面标注
混合定位 规则	文档对象问题页面 定位	文档对象问题文 本、PDF 页面文本	页面角色限制、文本匹配、候选评分	页面跳转与辅 助标注

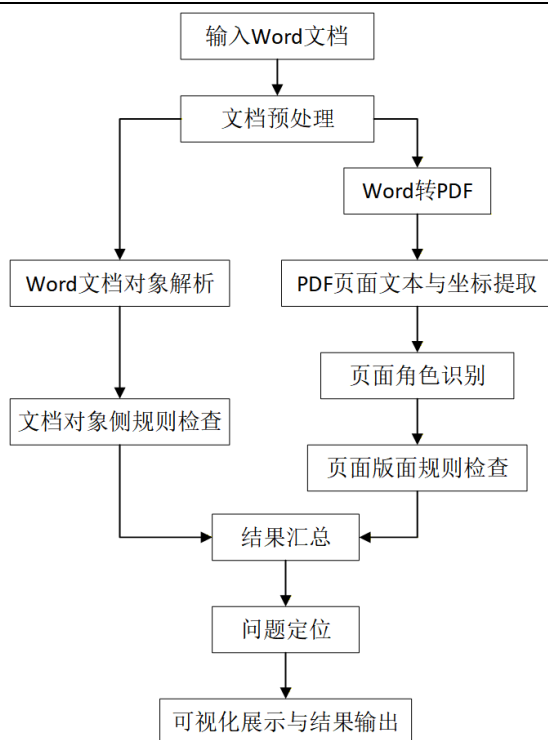


图 3-2 系统总体处理流程图

3.5 系统总体架构设计

结合系统处理流程，本文将系统划分为页面分析层、文档对象分析层和结果处理展示层三个部分。

3.5.1 文档对象分析层

文档对象分析层主要负责对 Word 文档进行结构化解析。该层通过文档解析模块提取论文中的各类板块、标题层级、段落文本以及格式属性信息，并将其组织成可供规则调用的上下文数据。随后，系统调用文档对象侧规则完成相关检查，生成相应的问题结果。

该层主要解决内容结构和样式属性类问题，是系统的基础检查层。

3.5.2 页面分析层

页面分析层主要负责对文档的页面表示进行处理。系统先将 Word 文档转换为 PDF，再从 PDF 中提取页面文本及其坐标信息。在此基础上，系统进一步进行页面角色识别，将页面划分为中文摘要、英文摘要、目录、正文、参考文献及其他后置部分等不同语义区域。随后，系统执行页面规则，对页码、页眉、章节起始、图题表题、公式编号等版面规则进行检查。

该层主要解决页面布局和视觉呈现类问题，是系统体现页面分析能力的关键部分。

3.5.3 结果处理与展示层

结果处理展示层负责对文档对象侧和页面分析侧的检查结果进行统一组织。该层一方面完成规则调度与结果合并，另一方面负责图形界面中的结果展示与交互。对于页面规则输出的问题，可直接根据页码和坐标进行标注；对于文档对象规则输出的问题，则通过页面角色限制和文本匹配方式实现近似定位，从而在图形界面中进行统一展示。该层使系统从“能检查”进一步提升到“能直观展示和辅助定位”的层次。图 3-3 展示了系统架构图，表 3-3 进一步说明了各层次模块之间的职责分工。

表 3-3 系统层次与模块职责表

层次名称	核心模块	输入	输出	主要职责
文档对象 分析层	文档解析、样式提取、.docx 规则执行	.docx 文件	结构化上下文、.docx 问题结果	完成结构识别与样式属性检查
页面分析 层	Word 转 PDF、文本坐标提取、页面角色识别、页面规则执行	PDF 页面 数据	页面问题结果、页码 坐标信息	完成版面类规则 检查
结果处理 展示层	规则调度、结果合并、定位与可视化	多来源问题结果	汇总结果、标注图像、 界面联动信息	统一组织结果并 提供交互展示

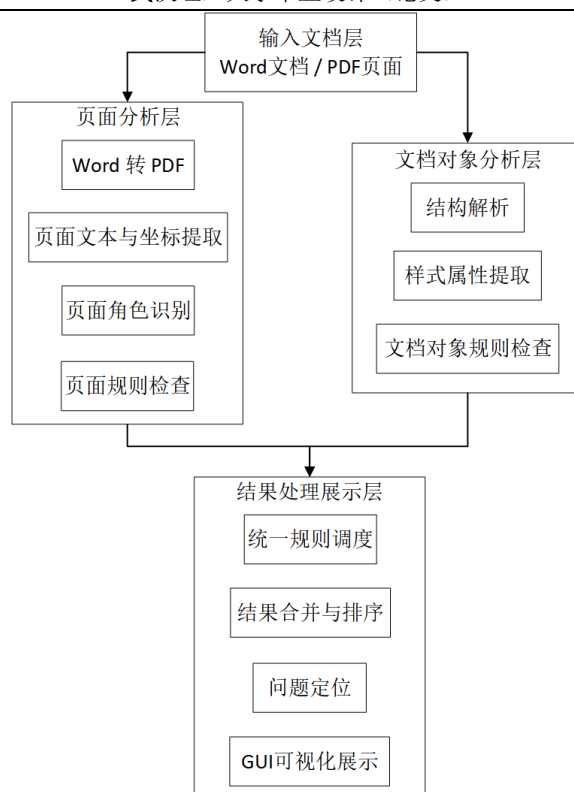


图 3-3 系统架构图

3.6 本章小结

本章围绕系统建设目标、功能需求、规则需求、总体处理流程和总体架构设计，对论文格式检查系统进行了整体分析。通过需求分析可以看出，毕业论文格式检查不仅涉及文档结构和样式属性，还涉及大量页面布局相关规则，因此单一依赖文档对象分析难以满足检查要求。在此基础上，本文设计了由页面分析层、文档对象分析层和结果处理展示层组成的系统总体架构，为后续系统详细设计与关键功能实现奠定了基础。

第4章 系统实现

4.1 文档对象分析实现

文档对象分析是整个系统的起点。程序在接收到用户选择的.docx 文件后，先通过 DocxContextBuilder 构建规则上下文，再由 DocxStructureAdapter 和 python-docx 对文档内容进行解析。这个过程不是简单地把段落文本读出来，而是要把后续检查需要用到的结构、样式和板块信息整理成统一的数据对象，供规则层反复调用。

从实际代码流程来看，这一阶段主要完成三件事。第一件事是识别文档由哪些部分组成，包括封面、摘要、目录、正文、参考文献、致谢等。第二件事是提取各部分中的段落内容、标题层级、运行块文本、表格和节对象信息。第三件事是把解析结果写入 RuleContext，让文档对象规则和混合检查流程都能直接使用。这样处理之后，规则实现不需要再次遍历全文，系统的整体结构也会更清晰。

4.1.1 文档上下文的构建

在代码实现中，HybridProcessor.process()并不会直接调用具体规则，而是先执行 _build_context()。该方法内部先通过 self.docx_builder.build(file_path)构建文档对象上下文，再根据是否存在同名 PDF 或用户指定 PDF，继续准备页面侧数据。这样设计的好处是，文档对象分析与页面分析能够共用同一份上下文信息，避免不同模块分别解析同一个文件。

DocxContextBuilder 的职责是把原始解析结果包装成 RuleContext。如果文档解析成功，就把 docx_obj、docx_sections 和文档部分顺序信息写入上下文；如果解析失败，则把异常信息写入 extras["docx_parse_error"]。后续规则在执行前会先判断这个状态，从而避免解析失败后继续进入检查逻辑。这种处理方式虽然不复杂，但对系统稳定性比较重要，尤其是在样本文档格式不统一时，可以避免整条检查链路直接中断。

4.1.2 文档结构与格式信息提取

在文档对象分析阶段，系统首先要解决“文档有哪些部分”的问题。当前实现会结合标题文本、板块关键词和段落顺序，将文档划分成摘要、目录、正文、参考文献等部分，并把各部分的段落内容分别保存。对于正文部分，还会继续提取标题列表、引文标识、参考文献条目、图题表题等信息。这样做的目的，是让后续规则不再面对一篇未分类的长文档，而是直接面向已经整理好的结构化数据。

除了结构划分外，系统还会提取与格式相关的属性。例如段落的对齐方式、行距、首行缩进、段前段后间距，运行块的字体、字号、上下标状态，以及节对象中的页边距、页眉页脚距离等。以页码、页眉和页脚相关规则为例，它们在 stage2_rules.py 中并不是只读取普通段落，而是会进一步访问节对象、页脚节点和 XML 信息。这样才能处理 Word 中常

见的自动页码、域代码和链接页眉页脚等情况。

4.1.3 样式继承与特殊对象处理

文档对象分析里最容易出问题的部分，其实不是读取文本，而是还原真实格式。因为 Word 中很多样式并不直接写在当前 run 上，而是来自段落样式、字符样式，甚至来自文档默认样式。如果只读取当前节点的显式属性，就很容易把样式继承漏掉。项目前期出现过英文标题 Abstract 被误报、正文中 python-docx 和 Open XML 被误报为“字体不是宋体”的情况，本质上都是这里处理得不够细。

针对这个问题，当前实现补充了样式继承回溯和字体候选项读取逻辑，并在字体检查时区分中文 run 和纯西文 run。也就是说，当规则要求是中文字体时，纯英文内容不会再被直接拿去和“宋体”“黑体”做硬匹配。这一改动对减少误报很有帮助，也让文档对象规则更符合真实排版习惯。

对于参考文献自动编号、页码域和页眉页脚这类对象，系统还继续下探到底层 XML。以 PageNumberFormatRule 为例，它并不是只检查“是否存在数字”，而是会调用 _footer_page_number_nodes() 读取页脚中真正的页码节点，再结合字体、字号、对齐方式进行判断。HeaderFormatRule 和 FooterFormatRule 也采用了类似做法，用来处理不同节之间可能存在的页眉页脚继承关系。这样一来，文档对象分析不仅能支持普通文本规则，也能覆盖 Word 中较难直接读取的对象。表 4-1 列出了这一阶段整理出的主要中间数据项。

表 4-1 文档对象中间数据项设计表

数据项名称	来源对象	主要字段	用途
摘要内容对象	摘要段落集合	文本内容、段落列表	摘要长度与摘要样式检查
关键词对象	关键词段落	标题文本、关键词列表	关键词数量与格式检查
标题列表对象	正文章节标题	标题文本、层级、顺序	标题编号与目录一致性检查
目录项对象	目录段落集合	条目文本、页码文本	目录一致性检查
参考文献对象	参考文献段落集合	编号、条目文本	编号顺序与引文一致性检查
页眉页脚节点对象	节对象头脚区域	节索引、文本、样式信息	页眉页脚与页码格式检查

4.2 页面分析与角色识别实现

页面分析模块体现了本课题“基于图像处理的 Word 文档格式检查”这一核心思路。系统不会只停留在 Word 对象层面，而是进一步把文档转换为 PDF，并把每一页当成一个带

有空间结构的版面来处理。虽然当前实现主要依赖 PDF 提取出的文本和坐标，尚未完全转向 OCR 图像识别，但其判断方式已经明显区别于单纯的文本比对，更接近人工查看排版时“先看位置，再看内容”的过程。

4.2.1 页面对象的构建

在混合检查流程中，如果系统能够找到用户指定的 PDF，或者在 Word 同目录下找到同名 PDF，就会进入页面分析阶段；否则会先执行 Word 转 PDF。对应的代码位于 `MainWindow.ensure_pdf_for_hybrid_check()` 和 `HybridProcessor.build_context()`。前者负责在界面层准备 PDF 输入，后者负责调用 `PdfExtractor.extract()` 将 PDF 解析为页面对象列表。

每一个页面对象中都包含页号、页面宽高、整页文本、文本 span 以及每个 span 的坐标框。这样做之后，页面侧规则就不需要直接操作原始 PDF 文件，而是只面对统一的数据结构。对后续的页码检查、页眉检查、题注检查和错误标注来说，这个页面对象都是共同的数据基础。

4.2.2 页面角色识别

页面分析的第一步不是立即检查格式，而是先判断当前页面属于哪一部分。当前项目在 `page_roles.py` 中定义了 `PAGE_ROLE_CN_ABSTRACT`、`PAGE_ROLE_EN_ABSTRACT`、`PAGE_ROLE_CATALOGUE`、`PAGE_ROLE_MAIN`、`PAGE_ROLE_BACKMATTER` 和 `PAGE_ROLE_OTHER` 六类页面角色，并通过 `build_page_roles()` 对整份 PDF 逐页标注。

这一过程主要依靠页面顶部标题、目录点线项、章节标题和正文后置部分标题进行识别。例如，如果页面中出现“目录”标题，且存在较多点线连接项和编号条目，就优先判为目录页；如果页面顶部更像“第 1 章”“第一章”“绪论”等正文起始形式，就把这一页及其后续页面纳入正文范围；如果页面顶部出现“参考文献”“致谢”或“附录”等标题，则把该页之后的内容划入正文后置部分。中文摘要与英文摘要则主要利用顶部标题“摘要”和“Abstract”来区分。

这个角色识别结果不仅会被页码和页眉规则复用，在 GUI 的错误定位中同样会用到。也就是说，系统后面无论是检查页码连续性，还是把“摘要错误”限制在摘要页范围内，本质上都依赖这一阶段的判断。因此页面角色识别虽然属于预处理步骤，但它对误报率影响很大。项目中出现过目录里的“第 4 章”被误识别成页码的问题，后续也是在这里不断收紧规则后才逐渐修正。

4.2.3 页面格式规则的实现

在页面角色确定之后，系统开始执行页面侧规则。与文档对象规则不同，页面规则更关注坐标、区域和相对位置。例如 `PageNumberPresencePdfRule` 会先调用

`_best_page_number_candidate()`在页面底端寻找页码候选，再结合页面角色判断这一页是否本来就应该有页码；`PageNumberBottomCenterPdfRule` 则进一步判断页码候选是否接近页面底端居中位置。

页眉检查的做法与此类似。`HeaderTopContentPdfRule` 会提取页面顶部文本，判断是否识别到规范页眉内容；`HeaderStartBoundaryPdfRule` 则会结合正文起始页，判断页眉是否从正文开始才出现。换句话说，页面规则不是单看某一页，而是会结合这页在整篇文档中的角色来判断它是否合理。

除了页码和页眉外，图题表题、章节另起页和公式编号位置等规则也依赖页面空间关系。以公式编号为例，规则并不是只检查“有没有(3-1)这样的编号”，而是还要判断编号是否位于公式行右侧末端；图题表题规则也不是只看是否出现“图”或“表”字样，而是要结合其与图表区域的上下关系进行判断。正因为这些规则面向的是页面最终呈现效果，所以它们更能体现本课题中“页面分析参与格式检查”的特点。表 4-2 汇总了页面规则的主要检测对象及判断依据。

表 4-2 页面规则设计说明表

规则名称	检测区域	主要特征	输出信息
页码存在性检查	页面底端区域	数字候选、页面角色、页码形式	缺失或异常页码提示
页码位置检查	页面底端中心区域	横向中心偏移、纵向位置	页码位置异常提示
页眉内容检查	页面顶部区域	规范页眉文本、正文页范围	页眉缺失或内容不符提示
页眉启用边界检查	正文起始页前后	页眉首次出现位置	页眉提前出现或缺失提示
章节另起页检查	正文页首区域	章节标题位置、分页边界	新章未另起页提示
图题表题位置检查	图表邻近区域	图表与标题上下关系	图题表题位置异常提示
公式编号位置检查	公式行末区域	公式主体与编号的横向关系	公式编号位置异常提示

4.3 规则调度与典型规则实现

这一节对应的是系统真正执行检查的主链路。前面两节分别解决了“Word 文档中有哪些对象”和“PDF 页面上有哪些版面信息”两个问题，而在实际运行时，还需要有一个中间

层把这两类信息组织起来，再交给不同规则分别检查。这个工作主要由 `model/hybrid/pipeline.py` 中的 `HybridProcessor` 完成。用户在界面中点击“文档检查”后，`MainWindow.hybrid_check()` 会先读取当前格式设置，再调用 `ensure_pdf_for_hybrid_check()` 准备 PDF 文件，最后把文档路径、PDF 路径和用户参数一并传入 `HybridProcessor.process()`。从这一刻开始，系统就进入统一的混合检查流程。

4.3.1 混合检查流程的组织

`HybridProcessor.process()` 的实现可以看成一条比较清晰的串行流程。程序首先调用 `_build_context()` 构建 `RuleContext`，其中既包含 `DocxContextBuilder.build()` 得到的文档对象信息，也会在 PDF 存在时通过 `PdfExtractor.extract()` 把页面文本、坐标框和页尺寸一起写入上下文。这样做的目的，是让后续规则不必再各自读取文件，而是都从同一个上下文对象中取数。结合项目实际来看，这种写法比较合适，因为摘要字数、关键词数量、标题字体等规则需要访问段落和 `run`，而页码位置、页眉内容、图题表题相对位置等规则又必须依赖页面坐标；如果没有统一上下文，后续规则很容易重复解析文件，也不利于结果合并。

上下文构建完成后，`process()` 会通过 `router.select()` 取得当前需要执行的规则集合，再按照 `docx` 和 `pdf` 两个引擎分别循环调用 `rule.check(ctx)`。这里并没有把所有规则混在一起顺序执行，而是先按照引擎分类，再把每个规则产生的 `Issue` 收集到 `grouped_issues` 中。这样处理有两个好处。一是便于保留规则来源，后面在界面中可以区分问题是来自文档对象检查还是页面分析检查；二是如果某条规则运行出错，程序会在 `except` 分支中构造一条“执行失败”的问题对象，而不会导致整次检查中断。对于毕业设计项目来说，这一点很重要，因为规则数量已经比较多，单条规则的健壮性不应该影响整条链路的可用性。

4.3.2 文档对象规则与页面规则的分工

从当前项目的代码来看，两类规则的职责划分已经比较明确。`create_default_registry()` 中注册的 `ChineseAbstractLengthRule`、`KeywordCountRule`、`CitationSuperscriptRule`、`ReferenceEntryFormatRule`、`ReferenceNumberSequenceRule`、`FontSizeFormatRule` 等规则，主要依赖 `python-docx` 提取出的结构和样式信息，适合放在文档对象侧完成。这类规则关注的是“内容属于哪一部分”“某段文字的字体字号是什么”“参考文献序号是否连续”这一类结构化问题，单靠页面外观不容易判断准确。

另一部分规则则更适合放在页面分析侧。例如 `PageNumberPresencePdfRule`、`PageNumberBottomCenterPdfRule`、`HeaderTopContentPdfRule`、`HeaderStartBoundaryPdfRule` 会重点使用页码候选框、页面顶部文本和页面角色信息；图题、表题、公式编号和目录展示相关规则则会使用文本块的上下关系、左右位置以及页面中的相对坐标。以这次项目中反复调整的页码检测为例，仅靠 `Word` 对象很难判断“数字是否真的位于页脚中部”，也无

法区分目录中的“第 4 章”和真正的页码数字，因此必须借助 `page_roles.py` 中的页面角色判断和 PDF 文本坐标分析。换句话说，文档对象侧解决的是“文档写了什么”，页面分析侧解决的是“这些内容最后排成了什么样”，两者缺一不可。

4.3.3 统一问题对象的构建与输出

各条规则执行结束后，`HybridProcessor` 会把结果交给 `IssueMerger.merge()` 统一合并，再通过 `issue_to_dict()` 转换成界面可以直接使用的字典结构。这个字典除了标题、描述、严重程度和规则编号以外，还会尽量保留 `page`、`bbox`、`metadata` 等字段。前两个字段直接服务于页面标注，`metadata` 则会继续保存板块名称、原始文本、引文 `marker`、参考文献条目等附加信息，供后面的定位函数使用。最后 `build_summary()` 会统计问题总数和不同严重程度的数量，`build_context_status()` 会记录本次是否成功使用了 PDF 页面上下文。这样处理后，界面层不需要了解每条规则的内部实现，只需要接收统一结果并负责展示即可。图 4-1 展示了这条从上下文构建、规则分流到结果回收的主流程。

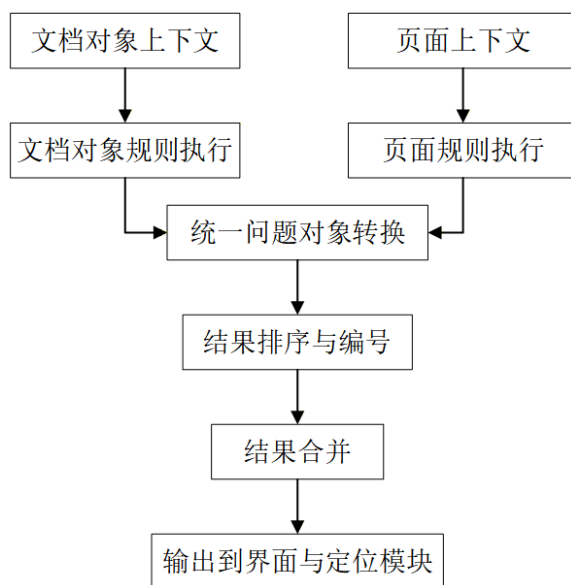


图 4-1 统一规则调度与结果合并流程图

4.4 问题定位与可视化实现

这一节更偏向系统实现层面的展示。对用户来说，最容易直接感受到的不是规则类名，而是“检查结果能不能看懂”“点了某条问题之后能不能快速找到页面上的位置”。因此本项目没有把结果只停留在文本列表上，而是在 `gui/main_window.py` 中补充了问题定位、页面跳转和矩形标注三部分逻辑。整个界面基于 `PyQt6` 实现，左侧用于显示 PDF 页面的可视化结果，右侧用于显示检查概览、问题列表和参数设置，界面中的按钮、列表和页码控件则通过信号与槽机制组织起来。

4.4.1 文件导入与检查触发

在交互入口上,主窗口提供了 `select_file()`和 `select_pdf()`两个方法,前者通过 `QFileDialog` 选择 Word 文档,后者用于手动补充 PDF 路径。考虑到项目题目强调基于图像处理的格式检查,而实际使用时用户并不一定会提前准备好 PDF,因此我在界面中增加了 `ensure_pdf_for_hybrid_check()`。该方法会优先使用用户手动指定的 PDF;如果没有,就调用 `docx_to_pdf_converter.convert()`自动把 Word 转成同名 PDF,并把生成路径重新写回界面。这样处理后,用户点击一次“文档检查”即可完成文档准备和规则执行,不需要再手动切换软件导出 PDF。

真正开始检查时, `hybrid_check()`会先调用 `get_format_settings()`读取右侧各个 `QComboBox` 当前选中的字体、字号和行距参数,再把这些参数作为 `user_formats` 传入 `HybridProcessor.process()`。如果 PDF 转换失败,程序会在 `RuntimeError` 分支里弹出 `QMessageBox` 提示,并自动退回到只执行文档对象检查的模式。这样写的好处是主流程仍然可用,不会因为某次转换失败就完全无法给出检查结果。

4.4.2 检查结果列表与页面联动

混合检查完成后, `display_hybrid_results()`会先清空旧结果,再把新返回的问题对象按严重程度和显示顺序重新整理,然后分别放入“检查概览”和“问题列表”两个区域。列表中的每一项并不是简单文本,而是由 `create_issue_item_widget()`构造的单独组件,里面会显示序号、板块名称和简短问题说明。这样做的目的,是让每条问题都成为一个可点击的交互单元,后面可以直接与左侧页面联动。

当用户点击右侧某一条问题时,程序会调用 `navigate_to_issue()`。这个方法会先检查当前问题是否已经带有明确页码;如果有,就直接把左侧页码控件 `page_spin` 跳到目标页。如果问题本身没有页码,但在可视化阶段已经补出了 `display_index` 对应的页面位置,也会优先使用补定位后的结果。与此同时, `populate_current_page_issue_buttons()`会把当前页涉及的问题同步列到左下方的小列表中,用户可以在页面内部继续切换同一页的多个问题。这种设计虽然不复杂,但实际使用时比只显示一长串报错文本清楚得多。

4.4.3 文档对象问题的近似定位

页面规则一般会直接输出 `page` 和 `bbox`,因此比较容易标注;真正麻烦的是文档对象规则。以摘要长度、关键词格式、引文上标和参考文献内容这类问题为例,它们本身是在 Word 结构上判断出来的,规则执行时通常拿不到 PDF 坐标,因此界面层还需要做一次“从 docx 问题回找 PDF 位置”的近似定位。这部分主要集中在 `collect_visual_issues()`、`locate_docx_issue_bbox()`、`locate_docx_marker_bbox()`和 `locate_reference_entry_boxes()`这几

个方法中。

具体实现时，我先利用 `build_pdf_page_roles()` 为每页分配角色，再通过 `allowed_roles_for_issue()` 和 `iter_issue_candidate_pages()` 缩小搜索范围。比如中文摘要相关问题优先在中文摘要页查找，目录问题限制在目录页，参考文献问题主要放在 `PAGE_ROLE_BACKMATTER` 对应的后置部分，正文中的引文上标则优先放在正文页处理。这样做的原因很直接，因为如果不先按板块过滤，像“摘要”“关键词”这样的短文本很容易在封面、目录或正文中反复误命中，前面调试时出现过的定位漂移问题，大多都和候选页过宽有关。

在候选页确定后，程序会调用 `group_page_lines()` 把同一水平带内的 `span` 合并成本行，再通过 `build_line_windows()` 或 `iter_page_windows()` 生成连续 1 至 4 行的窗口，参考文献条目则放宽到 6 行。`locate_docx_issue_bbox()` 会把问题文本归一化后，与这些窗口逐个比较，并综合考虑命中长度、额外字符数量、起始位置以及窗口行数给出一个匹配分数，最后选取得分最高的区域作为标注框。相较于最开始只按单行匹配的方式，这种多行窗口更适合处理正文长段落、换行标题和跨行参考文献条目。

对于局部标记更明显的问题，程序又单独写了两个分支。`locate_docx_marker_bbox()` 主要处理引文上标类问题，它会优先读取 `metadata` 中的 `marker`，例如[10]、[11]，然后直接到正文页的文本行里搜索这个标记，再把 `marker` 所在行整体框出。`locate_reference_entry_boxes()` 则用于参考文献内容问题，它会先按拆开多个条目，再优先用诸如[17]这样的编号去匹配后置部分的窗口。这样做虽然仍然属于近似定位，但比拿整段参考文献内容去全文搜索稳定得多，也更符合项目当前的实际效果。

4.4.4 错误标注框的绘制与缩放

定位结果确定后，左侧页面的绘制工作由 `render_pdf_page_with_issue_boxes()` 完成。该方法使用 `fitz` 打开 PDF，并通过 `page.get_pixmap()` 把目标页渲染成位图，再借助 `QImage` 和 `QPixmap` 转换成 `PyQt` 可以显示的图像对象。接着，程序使用 `QPainter`、`QPen` 和 `QColor` 在页面上绘制矩形框，并把“问题序号+板块名称”写到框的上方。为了让严重错误更醒目，代码中还根据 `severity` 区分了边框颜色和线宽。

缩放功能并不是重新渲染整份 PDF，而是由 `refresh_visualization_pixmap()` 根据当前显示模式重新缩放已经绘制好的 `QPixmap`。当用户选择“适应宽度”时，程序会读取滚动区域 `image_scroll.viewport().width()`，按照当前可视宽度进行等比缩放；当用户切换为百分比模式时，则使用 `display_zoom_factor` 对图片宽度进行重新计算。这样处理后，缩放操作只影响显示层，不会重复执行规则或重新搜索定位位置，界面响应会更加直接。结合 `update_issue_visualization_page()` 和 `scroll_to_issue()` 的联动逻辑，最终就形成了“右侧列表

点击一条问题，左侧自动跳页并显示标注框，当前页问题列表同步更新”的完整可视化链路。图 4-4 所示界面可以较直观地反映这一实现效果。

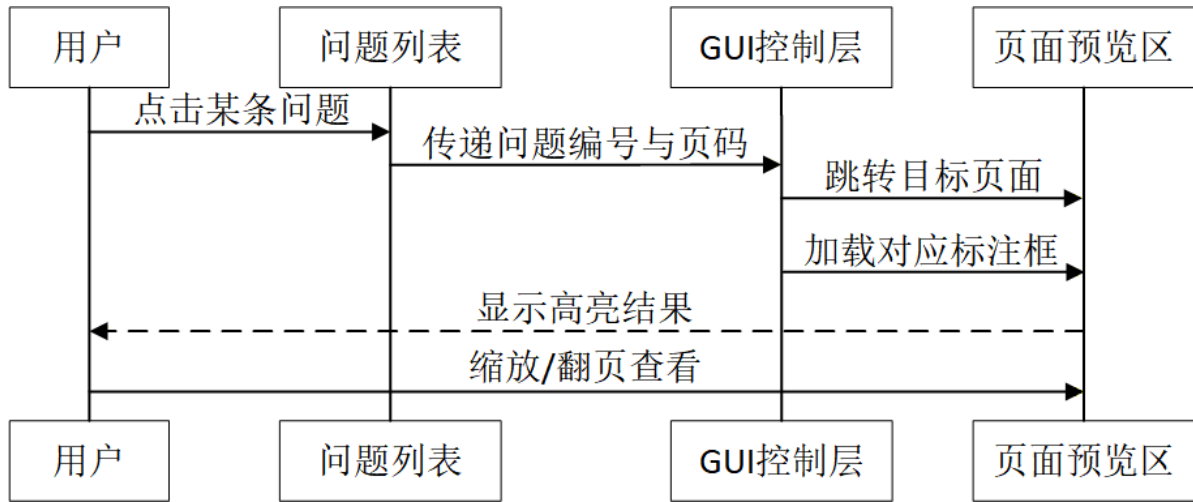


图 4-2 可视化标注交互时序图

问题定位模块是文档对象分析与页面版面识别之间的衔接层。对于页面规则输出的问题，系统可以直接根据页码和坐标框进行标注；对于文档对象规则输出的问题，则需要结合页面角色限制、关键词候选片段、多行窗口搜索和局部排序策略进行近似定位。也就是说，文档对象侧先判断“哪里有问题”，页面侧再尽量回答“问题大概出现在页面什么位置”。

在混合识别层面，系统并不是简单地把两类结果并排展示，而是通过统一的问题结构、统一的页面编号体系以及统一的可视化入口进行融合。用户点击某一问题条目后，界面自动跳转到对应页面，并在页面中高亮相关区域；若同一问题由多个子问题组成，还可以在同一页中依次显示多个标注框。这样，文档对象分析负责提供规则语义，页面版面识别负责提供空间位置，两者共同完成“问题解释+页面落点”的混合输出。

图 4-3 展示了系统主界面的左右分栏布局，建议保留左侧页面标注区与右侧问题列表的联动效果；图 4-4 展示了按钮操作区，用户可在这里选择需要检测的文件以及该文件对应的 PDF，如没有 PDF，系统将会生成对应的 PDF，以方便后续辅助检测。图 4-5 用于展示格式设置界面，建议突出页码、页眉页脚和正文参数设置区域。若正式排版时整张截图中文字偏小，可改用局部裁剪图提高可读性。

4.5 本章小结

本章围绕系统实现过程，对文档对象分析、页面分析、混合调度和界面可视化四个层面的核心做法进行了说明。其中，文档对象侧主要负责解析 Word 中的结构与样式，页面分析侧主要负责提取页面文本坐标并识别版面角色，混合调度层负责把两类规则组织到同一条执行链路中，而界面层则进一步完成结果列表、页面跳转和错误标注的联动展示。通

过这些实现，系统不仅能够发现格式问题，还能够把部分问题较准确地映射回页面位置，为后续测试与结果分析打下基础。

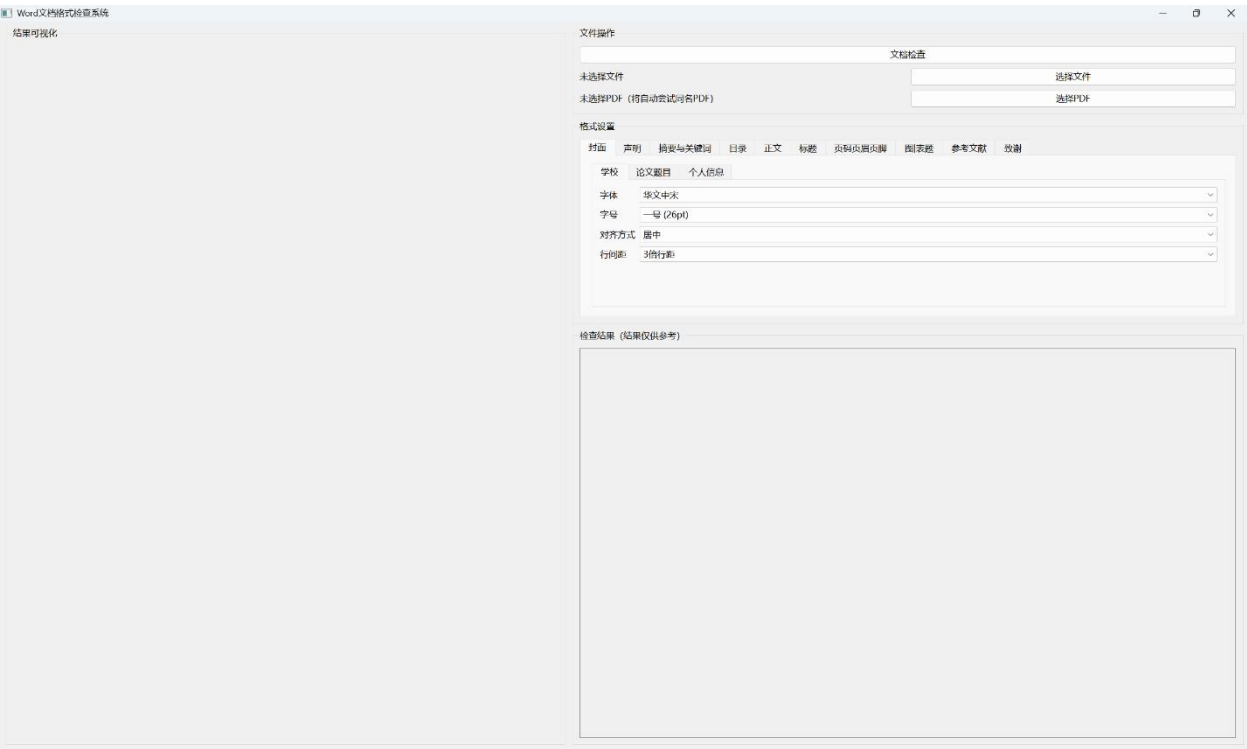


图 4-3 软件主界面截图



图 4-4 按钮操作区



图 4-5 格式设置界面截图

第5章 系统测试与结果分析

5.1 测试环境与测试方法

为了验证系统的可用性和检查效果，本文对已实现的论文格式检查系统进行了功能测试和结果分析。测试内容主要围绕文档对象侧规则、页面版面侧规则、混合结果展示与问题定位等几个方面展开。

测试环境以 Windows 平台为主，系统采用 Python 语言开发，图形界面基于 PyQt 实现，文档对象解析依赖 python-docx，页面分析基于 Word 转 PDF 后的页面文本与坐标提取结果完成。在测试过程中，主要以毕业论文格式样本文档及人工构造的异常样例作为测试对象，通过运行系统检查功能，观察问题输出是否符合文档实际情况，并对发现的误报和漏报进行分析。

本文系统测试主要在 Windows 平台下完成，开发语言与主要依赖库均与系统实现阶段保持一致，具体配置见表 5-1。

表 5-1 测试环境配置表

项目	配置内容
操作系统	Windows 11 64 位
开发语言	Python 3.12
图形界面框架	PyQt6
文档对象解析库	python-docx
PDF 处理与坐标提取	PyMuPDF (fitz)
运行与调试环境	VS Code
测试文档类型	.docx 论文样本、对应 PDF 样本、人工构造异常样本
主要测试内容	文档对象规则、页面规则、混合结果展示、问题定位与可视化

本文采用的测试方法主要包括以下三种。第一种是规则功能测试，即针对某一条或某一类规则，单独构造存在格式错误的样例，检查系统是否能够识别出对应问题。第二种是整体流程测试，即使用较完整的论文样例运行系统，观察从文档加载、规则执行到结果展示的整体过程是否顺畅。第三种是结果对照测试，即将系统输出与人工检查结果进行比较，分析系统的有效性以及当前仍存在的不足。

5.2 测试样本文档设计

为了保证测试结果具有一定代表性，本文在测试时使用了两类文档样本。一类是结构相对完整的论文样本文档，这类文档包含摘要、目录、正文、参考文献等典型板块，适合

用于验证系统整体处理流程和多规则协同效果；另一类是人工构造的异常样本，这类样本针对某一类规则故意设置错误格式，用于验证具体规则是否能够稳定触发。

在样本文档设计中，重点关注了以下几类典型问题：摘要长度不足或超出要求、关键词数量不符合要求、标题编号层级不正确、目录项与正文标题不一致、页码位置错误、页眉启用边界错误、章节未另起页、图题表题位置错误、公式编号未位于右侧行末、参考文献编号顺序错误等。通过覆盖这些常见问题，可以较全面地反映系统在毕业论文格式检查场景中的适用性。

此外，为了验证结果定位与页面标注功能，测试文档中还特意保留了部分能够在页面上明显定位的问题，例如页码位置异常、页眉缺失、参考文献条目错误等，以便观察系统在界面中的定位效果是否合理。

表 5-2 列出了测试样本文档的设计思路与覆盖范围。通过同时设置完整样本和异常样本，可以分别验证系统整体流程能力与单项规则触发能力。

表 5-2 测试样本文档与异常类型设计表

样本编号	样本类型	包含问题	测试目标
S1	完整论文样本	含摘要、目录、正文、参考文献、致谢等标准板块	验证系统整体流程、板块识别与多规则协同效果
S2	摘要异常样本	摘要长度不足、关键词数量不符合要求	验证摘要与关键词相关规则的识别能力
S3	标题异常样本	标题编号层级错误、标题标点不规范	验证标题编号与标题格式规则
S4	目录异常样本	目录项与正文标题不一致	验证目录一致性检查能力
S5	页码页眉异常样本	页码位置错误、页眉提前出现或缺失	验证页面规则对页码页眉的识别能力
S6	图表公式异常样本	图题表题位置错误、公式编号位置错误	验证图表与公式版面规则
S7	参考文献异常样本	编号顺序错误、引文与参考文献不一致	验证参考文献与引文一致性规则

5.3 文档对象侧规则测试结果

从测试结果看，文档对象侧规则在内容结构类问题和样式属性类问题上表现较为稳定。

对于摘要长度、关键词数量、参考文献编号顺序、引文标识、标题标点和标题层级等规则，系统能够较可靠地识别出明显不符合规范的情形。图 5-1 和图 5-2 分别展示了摘要姿势分析结果和参考文献分析结果。

摘 要

毕业论文格式检查是论文写作过程中的重要环节，但传统人工校对方式存在工作量大、效率低和易遗漏问题等不足。针对这一问题，本文设计并实现了一个基于 Python 的 Word 文档格式检查系统，用于辅助用户发现毕业论文中的常见格式错误。系统整体上以页面表示分析为主线，强调对文档最终版面效果的自动识别与检查。

本文首先结合毕业论文撰写规范，对论文中的格式规则进行分析，并将其划分为页面版面类规则和文档对象属性类规则两类。系统以 Word 转 PDF 后的页面文本与坐标分析为核心，对页码、页眉、章节分页、图题表位置信息和公式编号位置等问题进行判断；同时结合 Word 文档对象解析技术，对摘要、关键词、目录、标题、引文、参考文献以及字体、字号、对齐方式和行距等内容进行辅助检查。在此基础上，系统实现了统一规则调度、结果合并、问题定位和图形界面展示功能，使用户能够在查看问题描述的同时观察问题在页面中的大致位置。

测试结果表明，本文实现的系统能够较好地完成毕业论文中多数常见格式问题的自动检查，尤其在文档结构检查、样式属性检查、页眉页脚检查和结果可视化展示方面具有较好的实用效果。同时，系统在目录页码识别、复杂页脚结构处理和文档对象问题精确定位等方面仍存在一定不足，后续仍有进一步优化空间。

总体来看，本文完成了一个面向毕业论文格式检查场景、以页面分析为核心的原型系统实现，为毕业论文排版规范化提供了一种具有实际应用价值的辅助工具。

关键词：格式检查；Word 文档；页面分析；可视化定位



图 5-1 摘要字数分析结果展示



图 5-2 参考文献相关分析结果展示

在字体、字号、对齐方式和行距等样式检查方面，系统对中文摘要、英文摘要、目录、正文、参考文献等主要板块均具有较好的覆盖能力，并能够在结果中同时给出出错内容与错误描述。这样一来，检查结果不仅能够提示“存在问题”，还能够为用户指出问题大致所在的文本位置。

但在测试过程中也发现，文档对象侧规则对样式继承、特殊域代码以及复杂页脚内容

较为敏感。例如，在页码格式检查中，若页码对象的字号或字体并非直接定义于当前节点，而是通过样式链间接继承获得，则可能引发误判。针对这一问题，系统后续补充了样式链继承处理与页码域 run 信息提取逻辑，从而降低了页码字号误报的概率。这一现象表明，文档对象规则虽然适合完成样式属性检查，但在处理 Word 底层复杂结构时仍需结合具体实现细节持续修正和完善。

文档对象侧规则在测试中的表现见表 5-3。就整体情况而言，大多数规则在结构清晰、样式定义明确的样本中均能获得较好的检出效果。

表 5-3 文档对象侧规则测试结果表

规则名称	测试样 例数	正确检 出数	典型误差说明
摘要长度检查	6	6	对明显超长或过短情况识别稳定
关键词数量检查	6	6	当关键词字段结构清晰时结果稳定
标题编号层级检查	8	7	个别复杂编号样式下存在边界误判
标题标点检查	6	6	对标题末尾标点识别较准确
目录一致性检查	5	4	当目录项与正文标题存在格式差异时可能漏报
参考文献编号顺序检查	6	6	对连续编号错误识别较稳定
引文与参考文献一致性检查	5	4	个别上标格式不规范时匹配效果下降
字体字号检查	10	9	样式继承或域代码场景下偶有误判
对齐方式与行距检查	8	8	对普通段落样式问题识别效果较好

5.4 页面分析侧规则测试结果

页面分析侧规则主要面向页码、页眉、章节分页、图题表题和公式编号等版面类问题。从测试效果看，该部分规则较好地弥补了单纯文档对象分析的不足。尤其是在页码位置、页眉内容和章节另起页等规则上，系统能够直接利用页面文本坐标进行判断，因此所得结果更接近用户最终看到的论文排版效果。图 5-3 展示了图与图题分页情况下的文档分析检查结果，图 5-4 展示了下一章未另起一页的检查结果。

以页码检查为例，系统能够区分摘要部分与正文部分在页码形式上的差异，并进一步判断页码是否位于页面底端居中位置。在页眉检查中，系统可根据正文页顶部文本识别规范页眉是否存在，并检测页眉是否在第 1 章之前提前出现。在图题表题和公式编号检查中，系统也能够依据文本块之间的空间关系给出相应提示。



图 5-3 图与图题分置两页分析结果展示

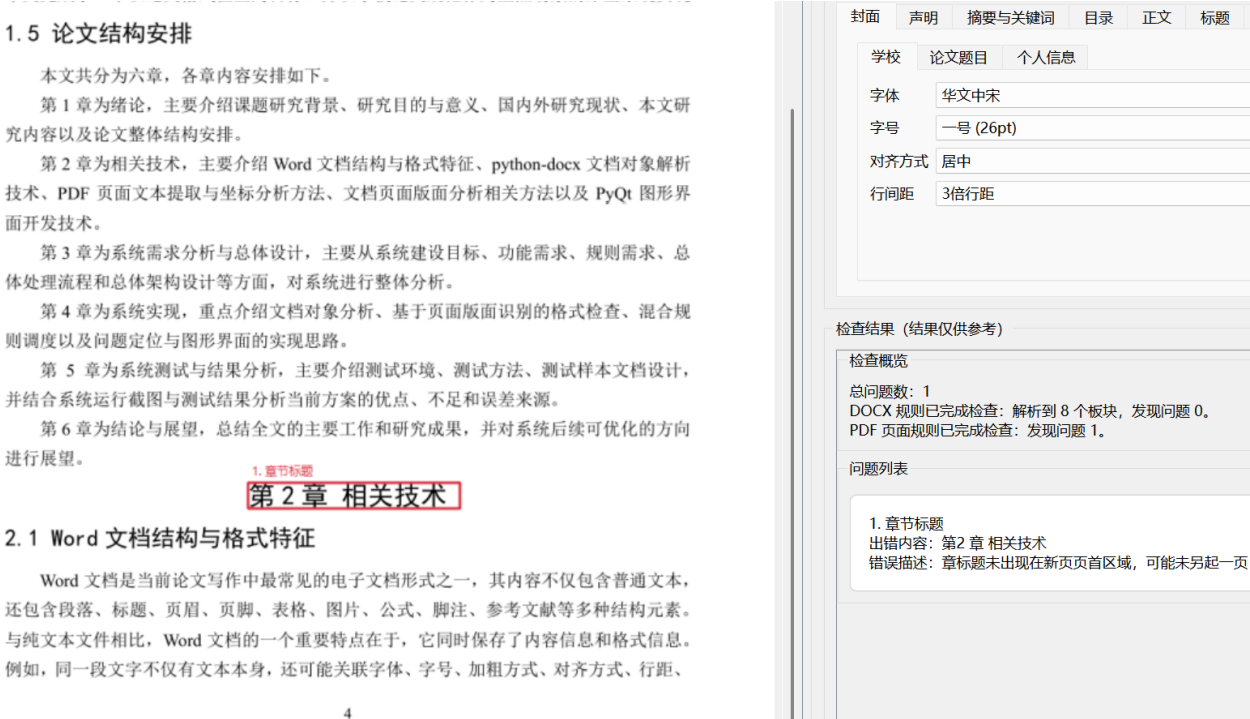


图 5-4 下一章未另起一页分析结果展示

不过，页面分析侧规则同样存在一定局限。测试过程中曾出现将目录页中的章节号或目录项页号误识别为页面页码的情况。其根本原因在于，目录页本身包含大量数字和点线

连接项，而这些数字在空间位置上与真正页码存在一定相似性。针对这一问题，系统后续对页码候选区域、候选排序方式以及目录页专用过滤条件进行了多次调整，使目录中靠右的条目页号不再轻易被识别为页面页码。经修正后，相关误报明显减少。

这说明页面规则虽然更适合处理版面类问题，但仍容易受到局部文本布局相似性的干扰，因此在实现过程中需要结合具体场景持续优化候选筛选条件。

页面分析侧规则在测试中的表现见表 5-4。整体而言，页面规则对典型版面问题具有较好的检测能力，但对数字密集和局部布局相似的页面仍需进一步优化。

表 5-4 页面分析侧规则测试结果表

规则名称	测试样例数	正确检出数	误报场景
页码存在性检查	8	7	目录页数字偶尔被识别为页码
页码位置检查	8	8	对底端居中偏移的识别较稳定
页码样式与连续性检查	7	6	罗马数字与阿拉伯数字混排时存在边界误差
页眉内容检查	6	6	正文页顶部文本识别效果较好
页眉启用边界检查	6	5	个别分页边界附近存在轻微误差
章节另起页检查	5	5	对新章起始页判断较稳定
图题表题位置检查	6	5	图表附近文本较密集时易受干扰
公式编号位置检查	5	4	公式与正文混排时存在误报可能

5.5 综合检查结果分析

从系统整体运行结果看，将页面分析与文档对象分析结合起来，能够比单一方式覆盖更多的格式检查需求。其中，页面分析侧规则更适合处理页码、页眉、章节分页、图题表题和公式编号等版面问题，文档对象侧规则则更擅长处理字体、字号、标题编号、引文和参考文献等结构与样式问题，两者在功能上具有较强的互补性。

在结果展示方面，系统能够将不同来源的问题统一汇总到同一结果列表中，并给出检查概览、问题条目和页面标注信息。与仅输出文字问题的方式相比，这种混合展示方式更有利于用户理解问题。例如，当用户看到“页码可能未位于页面底端居中”这类提示后，可以直接在页面预览中查看对应位置，从而快速确认问题是否成立。

同时，混合检查方式也体现出不同来源规则在精度表现上的差异性。文档对象规则通常在结构与样式属性问题上较为稳定，但在复杂底层对象场景下可能出现误报；页面规则在版面判断上更为直观，但在目录页、复杂文本块布局等场景下仍可能受到干扰。因此，

当前系统更适合定位为“辅助检查工具”，其主要作用在于帮助用户快速发现高概率问题，再结合人工判断完成最终确认。图 5-5 展示了系统对多来源问题进行统一汇总后的界面效果；图 5-6 展示了无格式问题的文档的检查结果，同时系统会提示所有检查链路都已正常执行。



图 5-6 综合检查结果界面截图

5.6 系统不足与误差分析

尽管当前系统已经能够完成较完整的毕业论文格式检查流程，但从测试结果来看，仍然存在一些不足和误差来源。首先，部分规则仍然依赖启发式判断，当面对格式较特殊或排版方式差异较大的文档时，可能出现误报或漏报，例如目录页中的数字分布、图题表题附近的复杂文本以及公式编号与正文混排等情况，都会影响页面规则的判断结果。其次，文档对象侧规则在处理 Word 底层复杂结构时仍存在一定不稳定因素，一些页码、页眉或页脚内容并不是以普通文本形式直接存在，而是通过域代码、文本框或样式继承实现，这会增加格式提取和准确判断的难度。

与此同时，文档对象问题的页面定位目前仍属于近似定位。虽然系统已经通过页面角色限制、文本归一化匹配和多行窗口搜索等方式提高了定位成功率，但当问题文本较短、重复度较高或页面内容较复杂时，定位结果仍可能不够精确。另一个较为明显的限制在于，当前系统页面分析链路主要依赖 PDF 文本与坐标提取，尚未接入真正意义上的 OCR 图像识别能力，因此在面对扫描版文档或文本提取质量较差的 PDF 时，页面分析效果仍会受到

较大影响。总体来看，这些不足并不影响系统作为论文格式辅助检查工具的基本使用价值，但也说明其在规则精度、复杂对象处理和定位可靠性等方面仍具有较大的优化空间。

测试中发现的典型误报与漏报案例归纳于表 5-5，这些案例反映出系统当前主要误差来源集中在复杂版式、特殊对象结构以及近似定位环节。

表 5-5 典型误报与漏报案例分析表

案例编号	问题类型	出现原因	当前处理方式	后续改进方向
C1	目录页页码 误报	目录项页号与真实页码 在空间位置上相似	调整候选区域与 目录页过滤条件	引入更细粒度页面 角色识别与候选评 分机制
C2	页码字号误 报	页码对象通过样式链继 承字号，未直接写入当 前节点	增加样式链继承 分析与域 run 提 取	完善底层 XML 解 析与继承回溯逻辑
C3	图题表题位 置误报	图表附近文本块较多， 局部空间关系复杂	收紧候选窗口与 关键词约束	增强局部版面结构 建模能力
C4	引文一致性 漏报	引文标识格式不统一， 上标形式存在差异	对常见 marker 形 式进行归一化	引入更鲁棒的引文 模式识别策略
C5	文档对象问 题定位偏差	候选页面文本重复度较 高，近似匹配精度不足	使用页面角色限 制与多行窗口匹 配	引入更稳定的跨行 匹配与排序模型

5.7 本章小结

本章围绕系统测试环境、测试方法、样本文档设计、文档对象侧规则测试结果、页面分析侧规则测试结果以及系统整体不足进行了分析。测试结果说明，当前系统已经能够较好地完成毕业论文中的多数常见格式检查任务，尤其在文档结构检查、样式检查、页码页眉检查和问题可视化展示方面表现较为稳定。与此同时，测试过程中暴露出的误报与定位误差问题，也为后续系统优化和功能扩展提供了明确方向。

第 6 章 结论与展望

6.1 论文工作总结

本文围绕毕业论文格式自动检查需求，完成了一个基于 Python 的 Word 文档格式检查系统原型。围绕学校毕业论文撰写规范，系统建立了从文档输入、规则检查到结果展示的完整处理流程，并重点面向论文中常见的结构、样式和页面版面问题开展自动分析。

在研究与实现过程中，本文完成了规则需求梳理、系统架构设计、关键功能开发以及测试分析等工作。其中，系统以页面分析为主要路径，对页码、页眉、章节分页、图题表题位置和公式编号位置等问题进行判断，并结合 Word 文档对象分析补充摘要、关键词、目录、标题、参考文献和引文等结构与样式类检查。通过引入问题定位与图形界面展示功能，检查结果能够以更直观的形式反馈给用户。整体来看，本文较好地完成了课题预期目标。

6.2 主要成果总结

本文的主要成果体现在三个方面。其一，完成了毕业论文格式检查系统的总体设计与开发，形成了较完整的软件处理流程。其二，构建了页面分析与文档对象分析相结合的检查机制，使系统既能处理摘要、关键词、标题、引文和参考文献等结构与样式问题，也能处理页码、页眉、章节分页、图题表题位置和公式编号位置等版面问题。其三，实现了问题定位与可视化展示功能，使不同来源的检查结果能够在同一界面中统一呈现，并便于用户快速确认问题位置。

在测试与调试过程中，系统还针对页码字号识别、目录页数字误识别为页码以及检查结果展示逻辑等问题进行了持续优化，使整体稳定性得到了一定提升。由此可见，本文实现的系统已经具备较为完整的毕业论文格式辅助检查能力，能够为用户提供具有实际价值的自动化支持。

6.3 不足与后续展望

虽然本文已经完成了系统的主要设计与实现工作，但受时间、经验和实现条件限制，当前系统仍存在一定不足，后续仍有进一步完善的空间。当前系统中的部分规则仍采用启发式实现方式，当文档排版形式较复杂或与常规论文样式差异较大时，仍可能出现误报或漏报；后续可以结合更多样本文档，对规则条件进行进一步细化和优化，以提高系统在复杂场景下的稳定性。

与此同时，文档对象分析在处理 Word 底层复杂结构时仍存在一定局限。页码、页眉、页脚等区域可能涉及域代码、文本框和样式继承关系，这些情况会影响系统对格式属性的准确提取，后续可以进一步加强对底层 XML 结构的分析能力，从而提高规则判断的可靠

性。当前页面分析链路主要依赖 PDF 文本与坐标提取，尚未真正接入 OCR 图像识别能力，因此当面对扫描版论文、文字提取质量较差的 PDF 或版式更加复杂的页面时，系统的分析能力仍然受限。后续可以考虑引入更完善的 OCR 方法或文档版面分析模型，以增强系统对复杂页面的适应能力。

此外，当前系统的问题定位仍以近似定位为主，尤其是文档对象侧问题在页面中的位置匹配还存在一定误差，后续可以从文本匹配策略、多区域候选排序以及跨页上下文分析等方面继续改进，提高问题标注的精确度。由于系统目前主要针对单一学校毕业论文规范进行实现，若后续能够将规则配置进一步参数化和模板化，则有望扩展到更多学校和更多文档规范场景中，提高系统的通用性与推广价值。总体而言，本文完成的系统已经为毕业论文格式自动检查提供了一个可运行的实现基础，未来若能在规则精度、页面识别能力、问题定位精度和规范适配范围等方面继续完善，其应用价值将进一步提升。

参考文献

- [1] 叶栩见. 基于边缘信息融合与多模态协同的文档版面分析方法[D]. 杭州电子科技大学, 2025.
- [2] 赵钰豪, 张仰森. 人工智能技术在文档解析中的应用综述[J]. 北京信息科技大学学报(自然科学版), 2025, 40(04): 90-98.
- [3] 王家宁, 朱磊, 张媛, 等. 基于深度学习的版面智能布局设计算法研究[J]. 印刷工业, 2023(04): 49-52.
- [4] 刘成林, 金连文, 白翔, 等. 文档智能分析与识别前沿: 回顾与展望[J]. 中国图象图形学报, 2023, 28(08): 2223-2252.
- [5] 蔡云冰, 杨词慧, 崔国昊, 等. 基于轻量化卷积神经网络的文档版面分析算法[J]. 南昌航空大学学报(自然科学版), 2024, 38(03): 45-52.
- [6] 张河锁. 复杂多场景文档版面分析方法的研究[D]. 华南理工大学, 2022.
- [7] 蔡云冰, 杨词慧, 崔国昊, 等. 基于参数重分配策略的文档版面分析方法[J]. 北京邮电大学学报, 2025, 48(02): 66-72+105.
- [8] Shehzadi T, Ifza I, Stricker D, et al. DocSemi: Efficient Document Layout Analysis with Guided Queries[C]. 2025 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW), 2025: 7536-7546.
- [9] 尹玲, 李家乐, 黄勃. 基于改进 YOLOv5s 的文档图像版面分析算法[J]. 软件导刊, 2025, 24(02): 146-154.
- [10] 袁沛愉, 陈亮, 刘昌宏. 融合多尺度特征的文档版面分析算法[J/OL]. 计算机技术与发展, 1-11[2026-03-01].
- [11] Minouei M, Soheili M R, Stricker D. Document Layout Analysis with an Enhanced Object Detector[C]. 2021 5th International Conference on Pattern Recognition and Image Analysis (IPRIA), Kashan, Iran, 2021: 1-5.
- [12] 马晓松. 基于多模态学习的文档版面分析技术研究[D]. 北方工业大学, 2025.
- [13] 谢海龙, 罗玮, 徐涛涛, 等. 基于复合注意力机制的文档版面分析算法[J]. 西安邮电大学学报, 2025, 30(03): 103-110.
- [14] Zhou E, Wu X, Xiao L, et al. Document Layout Analysis Via Positional Encoding[C]. 2022 IEEE International Conference on Image Processing (ICIP), Bordeaux, France, 2022: 1156-1160.
- [15] 谭伯乐. 基于大语言模型的文档要素检测和语法纠错研究与应用[D]. 广东工业大学, 2025.

致 谢

时光荏苒，岁月如歌。从论文选题到最终定稿，这段旅程凝聚了许多人的心血与支持。值此论文完成之际，我谨向所有给予我帮助与鼓励的人致以最诚挚的谢意。

首先，衷心感谢孙玉芬老师。从软件功能设计到代码实现，从论文框架搭建到细节打磨，孙老师始终以严谨的治学态度、渊博的专业知识和耐心的指导给予我悉心关怀。她多次在我遇到技术瓶颈时悉心点拨，帮助我理清思路、优化方案，更以精益求精的学术精神感染着我，让我深刻体会到科研工作的责任与乐趣。孙老师的教诲与鼓励，将成为我未来学术与职业道路上的宝贵财富。

其次，感谢武汉理工大学计算机与人工智能学院各位任课老师在本科阶段给予我的培养与帮助。通过相关课程的学习，我逐步掌握了程序设计、软件开发、文档处理等方面的基础知识，为本课题的完成奠定了必要基础。特别感谢辅导员老师在生活与学业上的对我的关心与支持。

同时，感谢并肩作战的同学们在课题研究和论文写作期间给予我的支持。大家在学习和实践中的讨论，使我在系统设计、功能实现和论文修改过程中获得了不少启发。

最后，感谢家人在整个学习过程中给予我的理解、鼓励与支持。父母多年来的辛勤付出与无条件的爱，让我能够专注于学业，你们的温暖与包容，是我永远的港湾。

再次向所有关心、帮助和支持过我的老师、同学和家人致以诚挚的谢意。